

CS120 Final Report

Yuecheng Xu, Yuhang Zhu
2021533010, 2021533173

Abstract

This report outlines our group’s project development process for the project of Computer Networks course, including the challenges we encountered and the solutions we implemented. It also shares our reflections on the course experience, suggestions for the teaching team to foster a more supportive learning experience, and advice for future students considering this course.

Contents

1	Our Struggle To This Project and Technical Solutions	2
1.1	Project 1	2
1.2	Project 2	3
1.3	Project 3	3
1.4	Project 4	4
2	Our Story And Self-Reflections	5
2.1	Yuecheng Xu	5
2.2	Yuhang Zhu	5
3	Suggestions For The Teaching Team	7
4	Advice For The Future Students	8
4.1	Key Insights and Recommendations	8
4.2	Language and Tool Recommendations	9
4.3	Final Thoughts	9
5	Conclusions	10

1 Our Struggle To This Project and Technical Solutions

The course project is a central component of CS120. We will now elaborate on each sub-project, detailing our problem-solving approach, the challenges encountered, and the solutions implemented.

1.1 Project 1

Project 1 requires us to implement the physical layer of a computer network, using sound for signal transmission and reception.

The first two tasks provided a startup framework, where we were required to familiarize ourselves with the ASIO interface, understand the basics of sound output and reception, and generate signals at specific frequencies. These tasks were relatively simple, and the teaching team provided reference code, which we simply had to understand and implement.

However, we encountered significant difficulties when working on Task 3. In this task, we needed to modulate the signal, transmit it through the air via sound, receive the signal, and then demodulate it into the correct format. Several challenges arose in this process:

Preamble Detection: If we solely used correlation (`syncPower`) to detect the preamble, we failed to locate the correct starting point for the payload, leading to the failure of the entire signal parsing. The main challenge here lies in the fact that the signal, after passing through the DAC, speaker, air gap, receiver's microphone, and ADC, is no longer the ideal signal. Differences, such as distortion, noise, and time shifts, occur along the way, which makes it difficult to accurately detect the preamble. We mimicked MATLAB to generate corresponding chirp signals and referred to the implementation in *Dhwani: Secure Peer-to-Peer Acoustic NFC* to generate the preamble. Unfortunately, neither approach significantly improved our monitoring results.

Modulation and Demodulation: We chose to use the MATLAB code provided by the teaching team, which implements a Binary Phase Shift Keying (BPSK) modulation scheme. However, we needed to make improvements, such as increasing bandwidth, baud rate, and bit rate. After facing a series of complex issues, we ultimately failed to achieve satisfactory results, only obtaining about 60% accuracy.

In the end, we were unable to successfully complete Task 3. Technically, the reason for this failure was that we did not adjust the correlation threshold correctly. The threshold varies across different devices and even different testing environments, making it difficult to tune. We should have used a different method to detect the preamble.

We also discovered an interesting fact: we initially used Java to complete the project and achieved only about 60% accuracy. However, after completing the check for Project 1, we switched to Go and rewrote the logic, using the exact same approach. This time, we achieved a much better accuracy rate (around 95%). We believe this is because Go's ASIO interface (Qjack) is more user-friendly and has lower latency and better interference resistance compared to Java's ASIO interface (JASIO). If we had used Go earlier, we would likely have been able to complete more tasks in Project 1 successfully.

1.2 Project 2

This project builds upon the results of Project 1, aiming to establish MAC layer using CSMA/CD protocol and ACK mechanism. Unlike the first project, where acoustic signals transmitted via air were used, this project simplifies the physical layer by directly connecting network devices through audio cables. These cables carry analog electrical signals thus providing a more stable and noise-resistant medium.

We regretfully report that only **Task 1** was completed. Despite this limitation, significant improvements were made in signal transmission compared to Project 1. Below, we elaborate on our progress and the challenges encountered:

In Project 1, our implementation struggled to achieve a 100% success rate in signal transmission. Even with the transition to audio cables in Project 2, the **receiver** component of our Java code initially failed to detect the preamble correctly. However, after revising our encoding and decoding strategies, significant progress was made:

- Because the two nodes are lined with cables, things get much easier without noisy. So we abandoned the BPSK modulation method and instead used a simplified approach of line coding, detecting sequences of five consecutive 1s or 0s, significantly improving signal reliability.
- The preamble sampling count was reduced from 480 to 48, decreasing frame size and increasing transmission speed while satisfying the time constraints of the system.

These changes resulted in substantial progress, allowing us to transmit data successfully and efficiently. However, the debugging and calibration of hardware components proved to be a time-consuming and frustrating process. Identifying specific bugs in our theoretical implementation hindered our progress. In contrast, the equivalent Go implementation from Project 1 easily achieved 100% signal transmission success.

During the course, we thoroughly understood the operational logic of the MAC layer and attempted to implement both an ACK mechanism and a CSMA listening mechanism in code. Though we have completed the key component in MAC layer, we faced a major challenge: synchronizing the MAC and physical layers specifically, how to manage the logic of the MAC layer while simultaneously handling transmission and reception tasks at the physical layer.

At that time, we were constrained by the mindset of using a single process, unaware that multi-threading could effectively solve this problem. Unfortunately, this limited approach during Project 2 restricted our ability to achieve better results. Now, we realize that leveraging multi-threading would have allowed us to handle these synchronization tasks more efficiently and seamlessly integrate the two layers.

1.3 Project 3

Since the later project depends on the previous one, we could not start it before the previous projects were done. Unfortunately, we didn't finish project 3.

However, we remained determined and did not give up. We carefully considered how we would build upon the completion of Project 2 to successfully tackle Project 3 and developed a solid understanding of its implementation details, including IP-layer data encapsulation, routing

mechanisms, Ethernet monitoring, hotspot handling, outbound traffic management, and seamless component integration.

Thanks to a classmate who provided open-source code written in Go for Project 3^[1], we were able to thoroughly study and analyze it. This meticulous review allowed us to continue making progress and prepare effectively for Project 4.

1.4 Project 4

Acknowledgments: We would like to express our sincere gratitude to Shunkang Jia for his foundational work on the Aethernet project, which provided a robust framework for acoustic link simulation and multi-layer protocol stack development. His comprehensive documentation and guidance greatly accelerated our progress and understanding. We use his code for Project3 as a base and build on our own Project4.

At this stage, we have built a basic yet functional network using audio signals. Above the IP layer lies a vast, rich, and open network realm. Before wrapping up the course project, adding DNS functionality to our network enhances its usability and aligns it with real-world network operations. This task revisits Project 3, where we implemented NAT and routing functionalities. By integrating DNS capabilities, we aimed to extend the network's functionality, enabling seamless domain name resolution and communication with external networks.

Our setup involved two nodes: Node1, a disconnected device requesting domain resolution, and Node2, a routing node with external network access and an integrated DNS server. When Node1 needed to resolve a domain, it sent a UDP packet containing the domain name as payload to Node2 via the Aethernet framework. Node2 parsed the packet, resolved the domain using its DNS server, and returned the resolved IP address in another UDP packet. Using the resolved IP, Node1 constructed an ICMP packet to ping the target address, simulating external communication.

The primary challenges in this project stemmed from the complexities of DNS resolution and integrating it seamlessly with the routing and NAT functionalities:

- **Simplified DNS Resolution:** Our implementation bypassed a hierarchical query structure typical of DNS servers. Instead, Node2 directly queried external DNS servers for domain resolution. While this approach expedited development, it diverged from the intended educational goal of familiarizing students with DNS query hierarchies.
- **Limited HTTP Implementation:** Due to time constraints and our reliance on a simplified DNS resolution method, we did not proceed with implementing HTTP functionalities. The HTTP task required a robust TCP handshake and sliding window mechanism, which posed additional complexities.

To align more closely with the course objectives and improve the project, future improvements include implementing a multi-level DNS query structure to better emulate real-world scenarios, adding HTTP functionalities to introduce concepts like TCP handshakes and protocol parsing, and incorporating enhanced testing metrics, such as Round Trip Time (RTT), for comprehensive performance evaluation.

2 Our Story And Self-Reflections

2.1 Yuecheng Xu

As a senior undergraduate student planning to focus on **robotics and deep learning** for my graduate studies, I initially chose this course because:

1. I wanted to understand the functioning of the **TCP protocol**, as many robots require TCP for control signal transmission.
2. I have a strong interest in computer networking concepts, such as **IP and DNS**.

However, I underestimated the **workload and complexity of the projects**, as well as the time commitment required. Completing each project often demanded around **six full days of effort**. This became overwhelming, especially considering my commitments outside the course, including completing a **graduation project at another institution**, participating in **corporate internships**, and handling multiple assignments and projects for other courses. The combination of responsibilities left me feeling mentally and physically exhausted.

Despite these challenges, I remained enthusiastic about the course, **attending every lecture and actively engaging with the material**. Although my project outcomes were less than ideal, I achieved valuable learning outcomes:

- Gained a solid understanding of computer networking concepts
- Learned the practical use of both **Java** and **Go programming languages**
- Enhanced my ability to handle complex code problems, improving my confidence in dealing with real-world “messy and dirty” systems

In conclusion, while the course presented significant challenges and difficult memories, I must acknowledge its **exceptional quality**. The thoughtful course design and thorough lecture preparation were evident throughout the term, making it a rewarding educational experience.

2.2 Yuhang Zhu

Similar to Yuecheng Xu, I had a comparable experience with course selection. Initially, I was uncertain about whether to take a course in computer networking, as my primary focus has always been in the field of **computer vision (CV)**. My decision to enroll in this course was heavily influenced by the advice of professors from the Systems and Security Center, who emphasized the importance of foundational computer science courses for CS students. This advice encouraged me to choose traditional courses like **Software Engineering, Compiler Principles**, and **Operating Systems**, all of which I performed well in and found immensely rewarding. These successes gave me the confidence to explore computer networking, despite my initial hesitation.

However, I underestimated the difficulty of the course, particularly the projects. Starting from scratch to build each project turned out to be far more challenging than I anticipated. While our theoretical understanding of the projects was sound, we were often met with practical obstacles that poured cold water on our enthusiasm. This was particularly discouraging during the first

project, whose deadline coincided with a critical period in my graduate school application process. Balancing these responsibilities while navigating the pressures of senior year left me mentally and physically exhausted, and unfortunately, I was unable to fully invest myself in the early stages of the course. This setback significantly hindered my ability to progress with subsequent projects, causing considerable academic anxiety.

Fortunately, I managed to regain my footing later in the semester. I am deeply grateful to my classmate **Shunkang Jia**, who consistently provided guidance and answered my questions. Additionally, the persistence and dedication of my teammate, Yuecheng Xu, motivated me to re-engage with the course. By the time we reached Project 4, I was able to independently complete our tasks and contribute meaningfully to our final project.

This course has been a transformative learning experience, and I have gained several key take-aways:

1. **Understanding Networking Fundamentals:** I developed a comprehensive understanding of the structure of computer networks, from the physical layer to the application layer, and the specific roles of various protocols.
2. **Improved Coding Skills:** My proficiency in both **Go** and **Java** improved significantly. The course's demanding projects required practical implementation of these languages, exposing me to advanced debugging techniques and real-world challenges.
3. **Practical Knowledge of Networking:** The course deepened my appreciation for the intricacies of networking, such as implementing routing algorithms, TCP sliding windows, and CSMA mechanisms. This hands-on experience has given me valuable insights into how theoretical principles are applied in real systems.

In retrospect, I see this course as a pivotal step in my academic journey. While the path was fraught with challenges, the rigor of the curriculum and the support from my peers and instructors helped me develop not only technical skills but also resilience and a deeper appreciation for computer networks. These experiences have strengthened my resolve to pursue further studies and have reaffirmed the importance of foundational courses in building a strong computer science education.

3 Suggestions For The Teaching Team

1. This course somewhat lacks communication between students and the teaching team, as well as among students themselves. Providing a more **formal platform**, such as a discussion board on **Blackboard** or **Piazza**, would allow students to raise questions encountered during learning. This approach would enable the teaching team or fellow classmates to offer timely solutions, better supporting those facing challenges.
2. Suggestion for Lectures: For key concepts, it would be helpful to provide repeated explanations and reinforcements. For instance, when discussing **TCP congestion control**, terms such as **advertised window**, **congestion window**, **max window**, and **cwnd** were clearly explained initially. However, beginners may forget these definitions and relationships as the lecture progresses, making it challenging to keep up with the content. Frequent reminders or recaps could greatly enhance understanding and retention.
3. Suggestion for Project Guidance: It would be helpful to remind students during Project 0 that each subsequent project may require **6 to 8 full days** of work. This information can assist students in making more informed decisions about course enrollment or withdrawal based on their time availability and commitment level.
4. Suggestion for Project Evaluation: The performance of Project 1 and Project 2 is significantly influenced by factors such as programming language, environment, and hardware. A single issue in any part can prevent students from scoring on an entire task, even if they have made considerable efforts and completed most of the task's code. To address this, it would be beneficial to offer partial credit (e.g., 1 point) if students can clearly explain their solution approach, walk through their written code, and demonstrate a correct understanding of the task, even if uncontrollable factors hinder optimal performance. This adjustment would better reflect their efforts and comprehension.
5. Maintain Existing Strengths: Timely responses in the WeChat group. Efficient communication with teaching assistants. Well-prepared instructors who demonstrate a deep understanding of the course content. These positive aspects significantly enhance the learning experience and should continue to be upheld.

4 Advice For The Future Students

The assessment of this course primarily consists of projects and exams. The projects are particularly challenging and original, often requiring considerations beyond the scope of the lectures, while the exams are of moderate difficulty. With proper review and attention during lectures, passing the exams is achievable. Homework assignments are relatively straightforward and serve as good practice for understanding the basics.

The course covers all stages of network layers, from the Physical Layer, MAC Layer, and IP Layer to the Transport Layer and Application Layer. These concepts are taught clearly in class, and the hands-on projects provide an opportunity to practice and solidify this knowledge. Throughout the semester, students engage in tasks such as modulating and demodulating sound waves, implementing CSMA mechanisms, understanding and constructing TCP sliding windows, creating IP/TCP/UDP packets, and designing forwarding tables, routing tables, and NAT. These tasks involve a high degree of debugging and iterative improvement, making time management a crucial factor for success. Completing these projects significantly enhances engineering and coding skills, making this course highly recommended for students interested in software engineering or information security.

4.1 Key Insights and Recommendations

Based on our experience, we offer the following recommendations for future students:

- **Start Projects Early:** Early projects, especially Project 1 and Project 2, form the foundation for later tasks. It is vital to begin working on these as early as possible. Hardware-level bugs, in particular, can be challenging to diagnose and resolve, as they are not always evident at the code level. Delays in early projects can cascade into significant challenges for subsequent ones.
- **Overcome Early Challenges:** Even if you cannot complete the early projects entirely, focus on implementing the foundational aspects. Doing so will make it easier to follow along with the rest of the course. Our failure to fully complete early projects caused significant stress and impacted our ability to progress smoothly.
- **Leverage Open-Source Resources:** Using open-source code as a starting point (with proper citations and acknowledgments) can significantly reduce the workload. While originality is encouraged, beginning with existing code allows for more time to focus on understanding and refining the projects.
- **Maintain Readable Documentation:** As you refine your code, pay attention to the readability of your README files. Clear documentation can be immensely helpful not only for future students but also for your own understanding as you revisit your projects.
- **Choose Programming Languages Wisely:** Selecting the right programming language is crucial. We strongly recommend using Go over Java. Go's concise syntax and efficient debugging processes significantly reduce the time and effort required. For example, while Java implementations often require around 5000 lines of code, equivalent functionality can be achieved in approximately 3000 lines using Go.

- **Consider Workload Management:** The workload for this course is substantial, especially when starting projects from scratch. It is advisable to limit the number of other courses taken during the same semester to balance the workload effectively.
- **Evaluate Course Fit:** The projects demand an in-depth understanding of detailed concepts. Students who are primarily looking for a general overview of computer networking or preparing for graduate school entrance exams may need to consider whether this course aligns with their objectives.

4.2 Language and Tool Recommendations

From our experience, Go programming combined with QjackCtl proved to be the most effective toolset for these projects. Go's efficiency in library integration and channel listening greatly simplified implementation and debugging. QjackCtl, which includes JASIO4ALL, offers a user-friendly interface and ensures stable device connections with lower latency. This combination was particularly advantageous during Project 1 and Project 2, where stability and efficiency were critical.

4.3 Final Thoughts

This course demands significant effort, especially in debugging and iterative development. For senior students, this effort is particularly important, as the course is not offered in the spring semester, and failing the class can have serious consequences. Despite the challenges, the hands-on projects provide invaluable experience, bridging theoretical knowledge and practical application. With better preparation, early project initiation, and effective collaboration, future students can navigate the course successfully and gain a profound understanding of computer networks. For those aspiring to pursue careers in software engineering or information security, this course is truly a must-try.

5 Conclusions

In this report, we share the methods used, challenges encountered, and solutions applied during the completion of our projects. Additionally, we reflect on our experiences, provide suggestions for the teaching team, and offer advice to future students considering this course. We were deeply impressed by the **dedication and meticulous work ethic** demonstrated by the teaching team, as well as the **innovative and well-structured course design**. As graduating seniors, this experience allowed us to truly witness what a high-quality course at ShanghaiTech University looks like, leaving us with a profound sense of admiration and appreciation. We put significant effort into preparing this report, offering our genuine thoughts and constructive suggestions with the hope of contributing to the continuous improvement of this excellent course. We sincerely hope that this course continues to improve and flourish, becoming even better over time. May ShanghaiTech University see the rise of more exceptional courses like CS120, fostering innovation and academic excellence for future generations.

References

- [1] Jiashun Kang. Aethernet: A network project. <https://github.com/jiashunkang/Aethernet>, 2025. Accessed: 2025-01-14.